

Regression Metrics and Losses: Placement Guide

Assume targets y_i , predictions $\hat{y}_i$, sample count n , mean target $\bar{y}$, and residual $e_i = y_i - \hat{y}_i$. Metrics are reported on a held-out validation/test set; losses are usually minimized during training.

MAE

1. Overview

Mean Absolute Error (MAE) is the average absolute prediction error. It is a robust, easy-to-explain regression metric used for demand, price, duration, and forecasting systems when an error of 10 units should count exactly twice as much as an error of 5.

2. Intuition

Ignore whether each prediction is high or low; measure its distance from truth. Predictions off by 2, 3, 5 have MAE $(2 + 3 + 5)/3 = 3.33$.

3. Prerequisites

- Mean, absolute value, residuals, NumPy arrays, train/validation/test splits.
- Basic regression and the distinction between a training loss and evaluation metric.

4. Core Concepts

Subtopic	Meaning and why it matters	Interview angle
Absolute residual	(	e_i
Linear penalty	Every extra unit costs one extra unit.	MAE is less outlier-sensitive than MSE.
Median optimum	A constant predictor minimizing MAE is the target median.	Contrast: MSE's constant optimum is the mean.

5. Algorithm / Working Process

1. Collect aligned y_true and y_pred .
2. Compute $abs(y_true - y_pred)$ per row.
3. Take the arithmetic mean; lower is better. During training, backpropagate a differentiable/subgradient implementation.

6. Mathematical Foundation

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

For one prediction, $\partial|e|/\partial\hat{y} = -\text{sign}(e)$ for $e \neq 0$; at zero it has a subgradient. Its constant-sized gradient is robust but may converge more slowly than MSE near the optimum.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import mean_absolute_error

y_true = np.array([100., 120., 90.])
y_pred = np.array([110., 115., 92.])
mae = mean_absolute_error(y_true, y_pred)
assert np.isclose(mae, np.abs(y_true - y_pred).mean())
print(f"MAE: {mae:.2f}")
```

8. Code Explanation

`mean_absolute_error` averages absolute elementwise residuals. The assertion is the direct NumPy definition and catches alignment or formula mistakes.

9. Training / Evaluation

Fit on training data; select models on validation MAE; report untouched test MAE with units (for example, ₹8,500). Use time-based splits for forecasts. Compare it with median-baseline MAE and inspect residuals by group.

10. Complexity and Cost

Evaluation is $O(n)$ time and $O(n)$ temporary memory (or $O(1)$ streaming state). CPU is sufficient; MAE itself adds negligible training cost.

11. Common Use Cases

- House-price or delivery-time prediction with understandable unit errors.
- Energy-demand and inventory forecasts with occasional bad readings.
- Model dashboards for business stakeholders.

12. Common Mistakes

- Comparing MAE across targets with different units/scales.
- Reporting train MAE as quality; this hides overfitting.
- Treating an error on ₹10 and ₹10,00,000 as equally important when relative error matters.
- Mismatching rows after joins or shuffling predictions independently.

13. Edge Cases / Limitations

MAE has no special penalty for catastrophic misses and is scale-dependent. Its non-smooth point at zero is usually handled by modern optimizers but can make optimization less smooth than MSE.

14. Variations

Variation	Change and use	Importance
Weighted MAE	Weight important samples more.	Practical placement topic.
Median absolute error	Use median rather than mean of absolute residuals.	More robust analysis.
Log-MAE	Evaluate errors after a log transform for positive long-tailed targets.	Project dependent.

15. Related Topics

MSE/RMSE penalize large errors more; MAPE expresses error as a percentage; Huber blends MAE's robustness with MSE-like smoothness; quantile loss estimates conditional quantiles rather than a central point.

16. Interview Questions

1. **What is MAE?** Average absolute residual.
2. **Is lower better?** Yes; zero is perfect.
3. **MAE vs MSE?** MAE penalizes linearly, MSE quadratically.
4. **Why robust to outliers?** A residual contributes proportionally, not squared.
5. **Why use absolute value?** Signed residuals cancel.
6. **What constant minimizes MAE?** The median.
7. **What are MAE units?** The same as the target.
8. **Can MAE be optimized?** Yes, with subgradients or built-in L1 loss.
9. **When is MAE a poor choice?** When rare large misses are especially costly.
10. **How handle unequal business value?** Use sample weights or a custom asymmetric loss.

17. Practice Tasks

- Implement MAE from NumPy without sklearn.
- Compare mean and median baselines on California Housing.
- Inject one extreme label and plot MAE versus MSE.
- Debug a pipeline whose MAE changes after sorting only predictions.
- Add sample weights for high-value customers.

18. Project Ideas

Project	Stack/data	Resume value
Delivery ETA estimator	pandas, scikit-learn; food-delivery trips	Business-unit error evaluation.
Used-car pricing	CatBoost/sklearn; used-car listings	Robust regression comparison.
Solar forecast dashboard	Python, FastAPI; weather/solar data	Production-style metric monitoring.

19. Quick Revision

- **Idea:** average absolute distance; **formula:** $\frac{1}{n} \sum |e_i|$.
- **Use:** typical absolute error matters and labels contain outliers.
- **Trap:** it does not punish large misses strongly; **one-liner:** "MAE reports the typical error in target units."

20. Final Cheat Sheet

Definition	I/O and steps	Pros / cons	Best use
Mean (	$y - \hat{y}$	)	two equal-length numeric vectors → residual, abs, mean

MSE

1. Overview

Mean Squared Error (MSE) averages squared residuals. It is a core regression training objective, especially when large errors are disproportionately harmful or a Gaussian-noise model is reasonable.

2. Intuition

Squaring makes an error of 10 cost 100, while an error of 2 costs 4. One disastrous prediction therefore dominates several small misses.

3. Prerequisites

- Residuals, powers, means, derivatives, linear regression, gradient descent.

4. Core Concepts

Subtopic	Meaning and why it matters	Interview angle
Squared residual	$e_i^2 \geq 0$ removes sign and amplifies magnitude.	Why does MSE react to outliers?
Smooth gradient	Derivative changes continuously with error.	Why is MSE convenient for neural nets?
Mean optimum	A constant MSE predictor is $\bar{y}$.	Link to least-squares regression.

5. Algorithm / Working Process

Compute residuals, square each, and average. Training updates parameters in the direction that reduces this mean; evaluation applies the same calculation on held-out data.

6. Mathematical Foundation

$$\text{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2, \quad \frac{\partial \text{MSE}}{\partial \hat{y}_i} = \frac{2}{n} (\hat{y}_i - y_i)$$

Minimizing MSE equals maximum likelihood when residuals are independent Gaussian with constant variance (up to constants/scaling).

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import mean_squared_error

y = np.array([3., -0.5, 2., 7.])
p = np.array([2.5, 0., 2., 8.])
mse = mean_squared_error(y, p)
assert np.isclose(mse, ((y - p) ** 2).mean())
print(f"MSE: {mse:.3f}")
```

8. Code Explanation

The squared residual expression is the definition. `mean_squared_error` provides the tested equivalent and supports weights/multioutput settings.

9. Training / Evaluation

Standardize features, not necessarily the target; if you scale `y`, inverse-transform predictions before reporting business MSE/RMSE. Monitor validation MSE and compare against a mean baseline. A widening train-validation gap indicates overfitting.

10. Complexity and Cost

$O(n)$ evaluation, $O(1)$ streaming memory. Squaring and gradients are cheap; outliers can cause large gradients, so target scaling/gradient clipping may help deep models.

11. Common Use Cases

- Linear regression and many neural-regression heads.
- Physical prediction where large deviations are unsafe.
- Gaussian-noise simulations and differentiable optimization.

12. Common Mistakes

- Calling MSE “average error” without noting squared units.
- Letting label outliers dictate the model unintentionally.
- Comparing MSE values across differently scaled datasets.
- Using it for categorical labels instead of cross-entropy.

13. Edge Cases / Limitations

MSE is highly outlier-sensitive and has squared target units, so it is less interpretable. Heteroscedastic noise violates its simple equal-variance interpretation.

14. Variations

Variation	Change and use	Importance
Weighted MSE	Weight residuals by importance or inverse noise variance.	Common practical extension.
Half MSE	$\frac{1}{2n} \sum e^2$, removes the 2 in gradients.	Math convention.
MSLE	Square errors in log1p space for nonnegative skewed targets.	Know its zero/negative limits.

15. Related Topics

RMSE returns MSE to target units. MAE is robust but non-smooth. R^2 rescales squared-error performance relative to a baseline. Huber caps the impact of large residuals.

16. Interview Questions

1. **Define MSE.** Mean squared residual.
2. **Why square?** Nonnegative objective and stronger large-error penalty.
3. **Units?** Squared target units.
4. **Which constant minimizes it?** The mean.
5. **Gradient?** $2(\hat{y} - y)/n$.
6. **Why popular in training?** Smooth, simple gradients.
7. **Outlier behavior?** A single extreme value can dominate.
8. **MSE vs RMSE?** RMSE is square root and interpretable in target units.
9. **Gaussian link?** It is the Gaussian negative log-likelihood objective with fixed variance.
10. **How mitigate outliers?** Validate labels, use Huber/MAE, or robust preprocessing.

17. Practice Tasks

- Derive the gradient of MSE.
- Train linear regression with gradient descent and verify decreasing MSE.
- Add an outlier and compare MSE, MAE, Huber.
- Compare unweighted and weighted MSE on imbalanced segments.
- Explain why scaling targets changes MSE by a square factor.

18. Project Ideas

Project	Stack/data	Resume value
Energy load forecaster	sklearn/PyTorch; hourly electricity	Regression-loss experimentation.
Battery health estimator	PyTorch; battery cycle data	Safety-sensitive large-error analysis.
Rental value predictor	XGBoost; property listings	Baselines and residual diagnostics.

19. Quick Revision

- **Formula:** $\frac{1}{n} \sum e^2$; **use:** large errors deserve large penalties.
- **Trap:** it is in squared units and is outlier-sensitive.

- **One-liner:** "MSE is smooth least squares with quadratic error cost."

20. Final Cheat Sheet

Definition	I/O and steps	Pros / cons	Best use
Mean squared residual	vectors → subtract, square, mean	smooth, differentiable / outlier-sensitive, squared units	Gaussian-like noise; severe misses

RMSE

1. Overview

Root Mean Squared Error (RMSE) is the square root of MSE. It retains MSE's emphasis on large errors while expressing results in the target's original units.

2. Intuition

It is a "typical error" score that makes large misses count extra, then converts the answer from squared units back to normal units. If MSE is 25°C², RMSE is 5°C.

3. Prerequisites

- MAE, MSE, square roots, residuals, and target units.

4. Core Concepts

Subtopic	Meaning and why it matters	Interview angle
Root transform	$\sqrt{\text{MSE}}$ restores units.	Does RMSE remove outlier sensitivity? No.
Monotonic equivalence	Same fixed dataset: lower MSE iff lower RMSE.	Can model ranking differ? Not on the same unweighted data.
Error scale	Compare RMSE with domain tolerance or target standard deviation.	Is RMSE=10 good? Only with context.

5. Algorithm / Working Process

Calculate residuals, square and average them, then take one square root. For model selection on the same split, MSE and RMSE choose the same winner.

6. Mathematical Foundation

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2} = \sqrt{\text{MSE}}$$

The square root is monotonic, but its gradient includes $1/(2\sqrt{\text{MSE}})$; training commonly minimizes MSE directly for simpler gradients.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import root_mean_squared_error

y = np.array([10., 20., 30.]); p = np.array([12., 18., 33.])
rmse = root_mean_squared_error(y, p)
assert np.isclose(rmse, np.sqrt(np.mean((y - p) ** 2)))
print(f"RMSE: {rmse:.2f} units")
```

8. Code Explanation

`squared=False` asks scikit-learn for RMSE. The NumPy assertion makes the MSE-then-root sequence explicit.

9. Training / Evaluation

Usually train with MSE and present RMSE to stakeholders. Evaluate after reversing any target transform. Report MAE alongside RMSE: a large RMSE-to-MAE gap signals a tail of large errors.

10. Complexity and Cost

$O(n)$ time; one final square root is negligible. No GPU is needed for metric calculation.

11. Common Use Cases

- Temperature, sales, load, and price forecasts needing target units.
- Kaggle-style regression leaderboards.
- Comparing models when large deviations matter.

12. Common Mistakes

- Claiming RMSE is robust because of the root; the squaring happens first.
- Averaging separate batch RMSEs instead of aggregating squared errors.
- Forgetting to inverse-transform scaled targets.
- Comparing across datasets with different target scales.

13. Edge Cases / Limitations

RMSE is still dominated by outliers and can hide which examples failed. It is zero only for perfect predictions and cannot express directional bias.

14. Variations

Variation	Change and use	Importance
NRMSE	Divide RMSE by range, mean, or standard deviation.	State denominator clearly.
RMSLE	RMSE in log space.	Positive skewed targets.
Weighted RMSE	Weighted MSE followed by root.	Important for unequal costs.

15. Related Topics

MSE is RMSE before the root; MAE gives linear errors; R^2 measures relative variance explained; MAPE avoids units but has zero-target problems.

16. Interview Questions

1. **Formula?** $\sqrt{\frac{1}{n} \sum e_i^2}$.
2. **Units?** Same as target.
3. **RMSE vs MSE ranking?** Identical on identical data.
4. **Does root make it robust?** No.
5. **Why report it?** Interpretability plus large-error sensitivity.
6. **Why train with MSE?** Simpler smooth objective.
7. **RMSE vs MAE?** RMSE emphasizes large misses.
8. **Can it be negative?** Never.
9. **How normalize it?** Divide by a documented scale statistic.
10. **Batch aggregation pitfall?** Sum squared errors/count, then root once.

17. Practice Tasks

- Compute RMSE manually and with sklearn.
- Show two models with similar MAE but different RMSE.
- Implement correct streaming RMSE.
- Compare RMSE before/after inverse transforming labels.
- Add normalized RMSE to a regression report.

18. Project Ideas

Project	Stack/data	Resume value
Air-quality forecast	pandas, LightGBM; city sensors	Unit-aware evaluation.
Taxi-fare predictor	sklearn; NYC taxi sample	Tail-error analysis.
Server load forecast	PyTorch; metrics logs	Monitoring-friendly error metric.

19. Quick Revision

- **Idea:** square errors, average, root; **formula:** $\sqrt{\text{MSE}}$.
- **Use:** need target units but care about severe misses.
- **Trap:** root does not neutralize outliers; **one-liner:** "RMSE is MSE's penalty in interpretable units."

20. Final Cheat Sheet

Definition	I/O and steps	Pros / cons	Best use
Square root of mean squared residual	vectors $\rightarrow$ MSE $\rightarrow$ root	target units, tail-sensitive / outlier-sensitive	forecasts with tolerated unit error

R² Score

1. Overview

The coefficient of determination, R^2 , measures how much better squared-error predictions are than always predicting the test-set mean. It is common for communicating regression fit, but is not an absolute error metric.

2. Intuition

Predicting every house price as the average is a baseline. $R^2 = 0.70$ means the model reduces squared error by 70% versus that baseline on the evaluated data.

3. Prerequisites

- Mean, variance, sums of squares, MSE, and regression baselines.

4. Core Concepts

Subtopic	Meaning and why it matters	Interview angle
SST	$\sum (y_i - \bar{y})^2$: total target variation.	What does the denominator represent?
SSE	$\sum (y_i - \hat{y}_i)^2$: unexplained squared error.	Why can test R^2 be negative?
Baseline-relative score	$1 - SSE/SST$.	$R^2=0$ equals mean-baseline performance.

5. Algorithm / Working Process

On one evaluation set, find its target mean, calculate SSE and SST, then return $1 - SSE/SST$. Use a test mean implicitly through library implementations; never mix train and test labels arbitrarily.

6. Mathematical Foundation

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Perfect prediction gives 1; mean-baseline prediction gives 0; a model worse than that baseline gives a negative value. With an intercept evaluated in-sample under OLS, $R^2 \in [0, 1]$; generally it need not be.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import r2_score

y = np.array([2., 4., 6., 8.]); p = np.array([2.2, 3.7, 5.8, 8.3])
r2 = r2_score(y, p)
manual = 1 - ((y - p) ** 2).sum() / ((y - y.mean()) ** 2).sum()
assert np.isclose(r2, manual)
print(f"R²: {r2:.3f}")
```

8. Code Explanation

The manual code forms SSE and SST explicitly. `r2_score` handles standard edge cases and multioutput options.

9. Training / Evaluation

Use R^2 alongside MAE/RMSE, never alone. Validate on unseen data; cross-validation reports mean and spread. For time series, use chronological folds. A high R^2 can coexist with unacceptable absolute error if target variance is large.

10. Complexity and Cost

$O(n)$ time and constant streaming aggregates for sums; CPU-only and negligible cost.

11. Common Use Cases

- Regression reports and linear-model summaries.
- Comparing models on the same target and split.
- Explaining improvement over a naive mean baseline.

12. Common Mistakes

- Saying R^2 is “accuracy” or percent of individually correct predictions.
- Assuming it is always 0–1.
- Comparing R^2 across unrelated datasets/target distributions.
- Ignoring MAE/RMSE and residual diagnostics.

13. Edge Cases / Limitations

R^2 is undefined/ill-conditioned when all true targets are constant (SST=0). It can reward leakage and says little about calibration, bias, or business cost.

14. Variations

Variation	Change and use	Importance
Adjusted R^2	Penalizes added predictors.	Essential for classical regression interviews.

Variation	Change and use	Importance
Cross-validated R^2	Aggregate held-out fold scores.	Better generalization estimate.
Explained variance	Similar but not identical when residual mean is nonzero.	Know distinction.

15. Related Topics

MSE supplies SSE; adjusted R^2 adds a complexity penalty; correlation r is not generally R^2 , though in simple OLS with intercept, $R^2 = r^2$.

16. Interview Questions

1. **Define R^2 .** Fractional SSE reduction versus mean baseline.
2. **What does 1 mean?** Perfect prediction.
3. **What does 0 mean?** Same SSE as mean predictor.
4. **Can it be negative?** Yes, especially out of sample.
5. **Is it accuracy?** No.
6. **Why pair it with RMSE?** R^2 is relative; RMSE gives units.
7. **What is SST?** Total squared deviation from target mean.
8. **When is it undefined?** Constant `y_true`.
9. **Does adding features always improve training R^2 ?** It cannot decrease for nested OLS models.
10. **How avoid misleading R^2 ?** Holdout/CV, leakage checks, absolute metrics.

17. Practice Tasks

- Calculate R^2 by hand for five rows.
- Build a mean baseline and verify $R^2=0$.
- Produce a negative R^2 example.
- Compare two datasets with identical RMSE but different R^2 .
- Report fold-wise R^2 with confidence-style spread.

18. Project Ideas

Project	Stack/data	Resume value
Student-score regression	sklearn; UCI student data	Baseline-relative reporting.
Real-estate modeling report	statsmodels/sklearn; Ames Housing	Interpretable model evaluation.
Crop-yield predictor	XGBoost; weather/yield data	CV and residual analysis.

19. Quick Revision

- **Formula:** $1 - SSE/SST$; **use:** fit relative to mean baseline.
- **Trap:** negative is possible and high R^2 is not low real-world error.
- **One-liner:** " R^2 is squared-error improvement over predicting the mean."

20. Final Cheat Sheet

Definition	I/O and steps	Pros / cons	Best use
Baseline-relative squared-error score	vectors $\rightarrow$ SSE, SST, ratio	scale-free, familiar / not absolute error, unstable for constant targets	compare regressors on one problem

Adjusted R^2

1. Overview

Adjusted R^2 modifies R^2 by penalizing predictors that do not earn their complexity. It is mainly used for classical linear regression feature selection and model summaries.

2. Intuition

Plain training R^2 likes extra knobs: adding a useless feature can slightly fit noise. Adjusted R^2 asks whether that tiny fit gain justifies spending a degree of freedom.

3. Prerequisites

- R^2 , sample size n , predictor count p , degrees of freedom, linear regression.

4. Core Concepts

Subtopic	Meaning and why it matters	Interview angle
Degrees of freedom	Residual freedom is $n - p - 1$ with intercept.	Why require $n > p + 1$?
Feature penalty	More predictors increase denominator penalty.	Can adjusted R^2 decrease? Yes.
Nested comparison	Most meaningful between models on same data/target.	Is it a substitute for CV? No.

5. Algorithm / Working Process

Fit a regression model, compute R^2 on its data, count fitted predictors (not the intercept), then apply the adjustment. Prefer validation/CV for final model choice.

6. Mathematical Foundation

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

For $p = 0$, adjusted R^2 equals R^2 . It rises only when a new variable improves fit enough to overcome the p -dependent penalty.

7. Practical Implementation

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1, 0], [2, 1], [3, 0], [4, 1], [5, 0]], dtype=float)
y = np.array([2, 4, 6, 8, 10.], dtype=float)
model = LinearRegression().fit(X, y)
r2, n, p = model.score(X, y), len(y), X.shape[1]
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
assert n > p + 1
print(f"Adjusted R²: {adjusted_r2:.3f}")
```

8. Code Explanation

`model.score` returns R^2 for regressors. `X.shape[1]` is the predictor count; the assertion protects the formula's required positive residual degrees of freedom.

9. Training / Evaluation

Use it for exploratory linear models, but choose final performance using held-out MAE/RMSE/ R^2 . High-dimensional or regularized models need CV and regularization rather than feature-count penalty alone.

10. Complexity and Cost

After R^2 , computation is $O(1)$. Training cost belongs to the underlying model, not the adjusted metric.

11. Common Use Cases

- Multiple linear regression reports.
- Comparing interpretable candidate feature sets.
- Teaching degrees of freedom and overfitting.

12. Common Mistakes

- Using number of one-hot source columns incorrectly after preprocessing.
- Counting intercept as a predictor in the standard formula.
- Selecting a final model only by in-sample adjusted R^2 .
- Applying it when $n \leq p + 1$.

13. Edge Cases / Limitations

It can be negative and is not a general predictive-performance guarantee. It does not test feature causality, remove multicollinearity, or replace AIC/BIC/CV.

14. Variations

Variation	Change and use	Importance
Predicted R^2	Based on prediction error (often PRESS).	Stronger generalization signal.

Variation	Change and use	Importance
AIC/BIC	Likelihood-based complexity penalties.	Important statistics extension.
CV R^2	Held-out fold R^2 .	Preferred project practice.

15. Related Topics

R^2 measures fit; adjusted R^2 addresses free feature additions; Lasso/Ridge control complexity during training; AIC/BIC score statistical likelihood with parameter penalties.

16. Interview Questions

1. **Formula?** $1 - (1 - R^2)(n - 1)/(n - p - 1)$.
2. **Why adjust R^2 ?** Plain training R^2 favors extra features.
3. **Can it decrease?** Yes, with weak features.
4. **Can R^2 decrease when adding OLS features?** Not in-sample for nested models.
5. **What is p ?** Number of predictors, excluding intercept.
6. **Minimum sample condition?** $n > p + 1$.
7. **Can it be negative?** Yes.
8. **Does it prevent leakage?** No.
9. **Adjusted R^2 vs CV?** CV estimates unseen performance; use CV for selection.
10. **Does it solve multicollinearity?** No.

17. Practice Tasks

- Add random columns and observe R^2 vs adjusted R^2 .
- Calculate it manually from an sklearn model.
- Compare feature subsets by adjusted and cross-validated R^2 .
- Trigger and handle $n \leq p+1$.
- Explain an adjusted R^2 decrease in a notebook.

18. Project Ideas

Project	Stack/data	Resume value
Salary regression study	statsmodels; salary survey data	Feature-selection reasoning.
Marketing spend analysis	pandas/statsmodels; ad spend data	Interpretable regression summary.
Insurance-cost model	sklearn; insurance charges	CV vs in-sample comparison.

19. Quick Revision

- **Idea:** R^2 minus a predictor-count penalty; **use:** classical feature-set comparison.
- **Trap:** still in-sample, not a replacement for CV.
- **One-liner:** "Adjusted R^2 rises only when a feature earns its degrees of freedom."

20. Final Cheat Sheet

Definition	I/O and steps	Pros / cons	Best use
Complexity-penalized R^2	$R^2, n, p \rightarrow$ apply adjustment	discourages useless features / not generalization proof	multiple linear regression

MAPE

1. Overview

Mean Absolute Percentage Error (MAPE) reports mean absolute error relative to the true value, usually as a percentage. It is popular for positive sales, traffic, and demand forecasts because **10% error** is intuitive across scales.

2. Intuition

Missing a sale by 10 units means little when actual sales are 10,000, but is huge when actual sales are 20. MAPE divides each absolute miss by its actual value.

3. Prerequisites

- MAE, division, percentages, positive target values, and zero handling.

4. Core Concepts

Subtopic	Meaning and why it matters	Interview angle
Relative error	(	e_i
Zero instability	Division by zero at $y_i = 0$.	Why is MAPE dangerous for sparse demand?
Asymmetry	Same additive error is weighted more for smaller actuals.	Can it bias forecasts? Yes.

5. Algorithm / Working Process

For each nonzero true value, divide absolute residual by its absolute true value, average ratios, then multiply by 100 for a percent. Define a business rule for zeros before computing it.

6. Mathematical Foundation

$$\text{MAPE} = \frac{100}{n} \sum_i \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

It is undefined at $y_i = 0$ and becomes enormous near zero. Do not silently replace zeros with a tiny epsilon unless that policy matches the product decision.

7. Practical Implementation

```
import numpy as np

def mape(y_true, y_pred):
    y_true, y_pred = np.asarray(y_true, float), np.asarray(y_pred, float)
    if np.any(y_true == 0):
        raise ValueError("MAPE is undefined when actual values are zero")
    return 100 * np.mean(np.abs((y_true - y_pred) / y_true))

assert np.isclose(mape([100, 200], [110, 180]), 10.0)
print(f"MAPE: {mape([100, 200], [110, 180]):.1f}%")
```

8. Code Explanation

Arrays enable vectorized computation. The explicit zero check prevents a deceptive finite/infinite output; choose WAPE, MAE, or a documented zero policy instead.

9. Training / Evaluation

Use chronological splits for forecasting. Evaluate MAPE only if zero/near-zero targets are rare and relative cost is appropriate. Segment it by product volume; report MAE/WAPE alongside it to avoid a few low-volume rows dominating.

10. Complexity and Cost

$O(n)$ time, $O(n)$ vector memory; trivial CPU cost. Training directly on MAPE can be numerically unstable near zero.

11. Common Use Cases

- Retail sales and demand forecasts with strictly positive volumes.
- Revenue forecast communication to non-technical teams.
- Comparing percentage accuracy across differently sized products.

12. Common Mistakes

- Applying MAPE with actual zeros or negative targets.
- Treating 1% error on a low-value item as equal business impact to 1% on a large account.
- Forgetting that many libraries return a fraction, not 0–100 percent.
- Optimizing MAPE blindly despite near-zero labels.

13. Edge Cases / Limitations

MAPE is undefined at zero, overweights small actual values, and is unsuitable for signed targets such as profit. It is not symmetric under swapping predictions and actuals.

14. Variations

Variation	Change and use	Importance
sMAPE	Denominator uses $ y $	y
WAPE	$(\sum$	e
MASE	Scale by naive forecast error.	Valuable time-series metric.

15. Related Topics

MAE measures absolute units; MAPE measures row-wise relative error; WAPE weights aggregate business volume; RMSLE is useful for positive multiplicative-scale errors but is not a percentage metric.

16. Interview Questions

1. **Formula?** Mean $|e/y|$ times 100.
2. **Why use it?** Interpretable relative error.
3. **Can actual be zero?** No, MAPE is undefined.
4. **Negative target?** Usually avoid; interpretation breaks down.
5. **MAPE vs MAE?** Percentage versus target units.
6. **Main bias?** Small actuals get huge weight.
7. **MAPE vs WAPE?** MAPE averages row percentages; WAPE aggregates errors/actuals.
8. **Should a model train on MAPE?** Only with careful positive, bounded-away-from-zero targets.
9. **What does 12 mean?** Average absolute percentage error of 12%.
10. **Alternative for zeros?** MAE, WAPE, MASE, or a domain-defined metric.

17. Practice Tasks

- Implement safe MAPE and test its zero exception.
- Show how one target near zero dominates MAPE.
- Compare MAPE and WAPE on imbalanced product sales.
- Build an item-level error dashboard with volume segments.
- Decide and document a zero-demand evaluation policy.

18. Project Ideas

Project	Stack/data	Resume value
Store-demand forecast	pandas, LightGBM; retail sales	Metric selection under zeros.
Subscription revenue forecast	sklearn; subscription cohorts	Percentage KPI reporting.
SKU forecast monitor	FastAPI/Plotly; inventory data	Segment-aware evaluation.

19. Quick Revision

- **Formula:** $100 \cdot \text{mean}(|e/y|)$; **use:** positive, nonzero targets and relative importance.
- **Trap:** zero and near-zero actuals.
- **One-liner:** "MAPE is intuitive percentage error, but it is fragile around zero."

20. Final Cheat Sheet

Definition	I/O and steps	Pros / cons	Best use
Mean absolute percentage residual	nonzero vectors $\rightarrow$ abs residual / actual $\rightarrow$ mean $\times 100$	scale-free, intuitive / undefined at zero, small-value bias	positive sales/demand forecasts

Huber Loss

1. Overview

Huber loss is a robust regression loss: it is quadratic for small residuals like MSE and linear for large residuals like MAE. It is useful when labels have some outliers but smooth optimization is still desired.

2. Intuition

Small misses are treated gently and precisely with a squared penalty. Once a miss exceeds threshold δ , Huber stops letting it explode quadratically—like a circuit breaker for bad labels.

3. Prerequisites

- MAE/MSE, piecewise functions, derivatives, residuals, gradient descent, robust statistics.

4. Core Concepts

Subtopic	Meaning and why it matters	Interview angle
Threshold δ	Boundary between quadratic and linear regions.	What happens as $\delta \rightarrow \infty$? MSE.
Inlier region	(	e
Outlier region	Large errors use δ	e

5. Algorithm / Working Process

1. Compute residuals.
2. Mark residuals within **delta** as inliers.
3. Apply squared loss to inliers and linear loss to outliers.
4. Mean/sum the losses and backpropagate; choose **delta** using validation/domain scale.

6. Mathematical Foundation

$$L_{\delta}(e) = \begin{cases} \frac{1}{2}e^2, & |e| \leq \delta \\ \delta(|e| - \frac{1}{2}\delta), & |e| > \delta \end{cases}$$

$$\frac{dL}{de} = \begin{cases} e, & |e| \leq \delta \\ \delta \operatorname{sign}(e), & |e| > \delta \end{cases}$$

It is continuous and differentiable at $|e| = \delta$. Smaller δ approaches scaled MAE; larger δ approaches MSE.

7. Practical Implementation

```
import numpy as np

def huber_loss(y, p, delta=1.0):
    e = np.abs(np.asarray(y, float) - np.asarray(p, float))
    return np.mean(np.where(e <= delta, 0.5 * e**2, delta * (e - 0.5 * delta)))

assert np.isclose(huber_loss([0, 10], [0, 2], delta=1), 3.75)
print(huber_loss([0, 10], [0, 2], delta=1))
```

8. Code Explanation

`np.where` selects the appropriate branch per residual without Python loops. The test covers both branches: zero error gives 0; error 8 gets linear-tail loss 7.5, averaged to 3.75.

9. Training / Evaluation

Scale the target or choose δ in target units. Tune `delta` with validation data; inspect label outliers before declaring them noise. Evaluate final predictions with business MAE/RMSE too—Huber is primarily a training objective.

10. Complexity and Cost

$O(n)$ time and vector memory. It is roughly as cheap as MSE, CPU/GPU friendly, and avoids unbounded MSE gradients from extreme residuals.

11. Common Use Cases

- Sensor measurements with sporadic faults.
- Financial/price regression with occasional erroneous labels.
- Object detection bounding-box regression (often Smooth L1/Huber variants).

12. Common Mistakes

- Using default $\delta = 1$ on a target measured in thousands without scaling.
- Assuming Huber automatically fixes data-quality issues.
- Reporting Huber loss as a business-unit error; it has hybrid units/meaning.
- Confusing Huber regression with Huber contamination models.

13. Edge Cases / Limitations

The threshold needs tuning and no fixed value is universally robust. If most data are true heavy tails, a richer distributional model or quantile objective may be more appropriate.

14. Variations

Variation	Change and use	Importance
Pseudo-Huber	Smooth approximation $\delta^2(\sqrt{1 + (e/\delta)^2} - 1)$.	Research/optimization detail.
Smooth L1	Framework-specific scaling convention.	Important in computer vision.
HuberRegressor	sklearn linear robust model.	Practical placement library knowledge.

15. Related Topics

Huber bridges MSE and MAE. Quantile loss gives asymmetric error costs; RANSAC discards outliers at the estimator level; robust scaling makes any loss less sensitive to feature extremes.

16. Interview Questions

1. **What is Huber loss?** Quadratic near zero, linear in tails.
2. **Why use it?** Smooth optimization with outlier resistance.
3. **What is delta?** Error threshold separating regions.
4. **Small delta limit?** MAE-like behavior (with scaling).
5. **Large delta limit?** MSE behavior.
6. **Is it differentiable at delta?** Yes.
7. **Why less outlier-sensitive?** Tail gradient magnitude is capped at delta.
8. **Where used in CV?** Bounding-box regression/Smooth L1.
9. **How tune delta?** Validation plus target/noise scale.
10. **Is it an evaluation metric?** It can be, but is most often a training loss.

17. Practice Tasks

- Implement Huber loss and verify continuity at δ .
- Plot MAE, MSE, Huber over residuals -5 to 5.
- Tune delta on data with injected outliers.
- Compare [LinearRegression](#) and [HuberRegressor](#).
- Explain delta in original and standardized target units.

18. Project Ideas

Project	Stack/data	Resume value
Sensor-fault-tolerant forecast	PyTorch; IoT temperatures	Robust-loss justification.
Robust property valuation	sklearn; Ames Housing	Outlier diagnostic workflow.
Bounding-box regressor demo	PyTorch/torchvision; Pascal VOC	CV loss knowledge.

19. Quick Revision

- **Idea:** MSE for small errors, MAE-like tail for big ones.
- **Formula:** piecewise at δ ; **use:** occasional label/sensor outliers.

- **Trap:** **delta** is scale-sensitive; **one-liner:** "Huber caps an outlier's gradient without losing local smoothness."

20. Final Cheat Sheet

Definition	I/O and steps	Pros / cons	Best use
Piecewise quadratic-linear loss	residual + $\delta \rightarrow$ branch, average	robust and smooth / threshold tuning	noisy regression; box regression

Quantile Loss

1. Overview

Quantile loss (pinball/check loss) trains a model to predict a conditional quantile, not only the conditional mean. It powers prediction intervals and asymmetric-cost decisions such as inventory safety stock.

2. Intuition

For $\tau = 0.9$, underpredicting is nine times as costly as overpredicting, so the model learns a value that 90% of outcomes fall below. That is useful when running out of stock hurts more than holding extra inventory.

3. Prerequisites

- Percentiles/quantiles, residuals, conditional distributions, piecewise loss, regression.

4. Core Concepts

Subtopic	Meaning and why it matters	Interview angle
Quantile τ	Desired coverage from 0 to 1.	What does $\tau = 0.5$ predict? Median.
Asymmetry	Under/overprediction receive unequal slopes.	How encode stockout cost? Choose high τ .
Prediction interval	Train low and high quantiles, e.g. 0.1/0.9.	Why can intervals cross? Separate models lack constraints.

5. Algorithm / Working Process

1. Choose τ from desired coverage/cost.
2. Predict $\hat{y}$, compute $e = y - \hat{y}$.
3. Penalize positive and negative residuals with different slopes.
4. Minimize average loss. For intervals, train at two or more quantiles and validate empirical coverage.

6. Mathematical Foundation

$$L_{\tau}(y, \hat{y}) = \max(\tau(y - \hat{y}), (\tau - 1)(y - \hat{y})) = \begin{cases} \tau e, & e \geq 0 \\ (\tau - 1)e, & e < 0 \end{cases}$$

Minimizing expected L_τ yields the conditional τ -quantile. At $\tau = 0.5$, it is proportional to MAE and predicts the conditional median.

7. Practical Implementation

```
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor

def pinball_loss(y, p, tau):
    e = np.asarray(y, float) - np.asarray(p, float)
    return np.mean(np.maximum(tau * e, (tau - 1) * e))

assert np.isclose(pinball_loss([10, 0], [8, 2], 0.9), 1.0)
model = GradientBoostingRegressor(loss="quantile", alpha=0.9, random_state=0)
print("alpha=0.9 model predicts an estimated 90th conditional percentile")
```

8. Code Explanation

`np.maximum` implements the two linear branches. `alpha=0.9` tells gradient boosting to optimize the 90th quantile; fit it with real `X`, `y` like any sklearn regressor.

9. Training / Evaluation

Use representative train/validation/test splits, especially chronological splits for forecasts. Evaluate pinball loss at the same τ , plus coverage: for a 0.9 quantile, roughly 90% of held-out actuals should be at or below prediction. Tune model capacity, not τ as a generic accuracy knob.

10. Complexity and Cost

The loss itself is $O(n)$. Training cost depends on model family; estimating several quantiles commonly requires multiple model heads/models, increasing compute and memory roughly with the number of quantiles.

11. Common Use Cases

- Demand planning and safety stock.
- Travel-time or delivery-time SLA estimates.
- Financial risk (Value at Risk) and weather uncertainty bands.

12. Common Mistakes

- Calling a 0.9 quantile a 90% probability the exact value occurs.
- Evaluating with MSE only instead of pinball loss and coverage.
- Assuming independently trained lower/upper quantiles never cross.
- Choosing τ without translating business costs/coverage.

13. Edge Cases / Limitations

The loss has a kink at zero and coverage can fail under distribution shift. Separate quantile models can cross; quantiles describe conditional percentiles, not complete uncertainty causes or causal confidence intervals.

14. Variations

Variation	Change and use	Importance
Median regression	$\tau = 0.5$, MAE-like.	Fundamental.
Multi-quantile model	Predict many τ values jointly.	Useful uncertainty output.
Quantile Huber loss	Smooth Huber-style quantile objective.	Deep RL/research, e.g. distributional RL.

15. Related Topics

MAE targets the conditional median; MSE targets conditional mean; Huber robustly targets a central value; conformal prediction can wrap models to calibrate finite-sample prediction intervals.

16. Interview Questions

1. **What does quantile loss optimize?** A conditional quantile.
2. **Formula?** $\max(\tau e, (\tau - 1)e)$.
3. **What is tau=0.5?** Median/MAE-like regression.
4. **What does tau=0.9 mean?** About 90% of conditional outcomes lie below predictions when calibrated.
5. **Why asymmetric?** Under/over errors have different costs.
6. **Use for inventory?** High quantile protects against stockouts.
7. **How evaluate?** Pinball loss and empirical coverage.
8. **Can quantiles cross?** Yes, with separately trained models.
9. **Mean vs median vs 90th?** MSE vs MAE vs quantile loss targets respectively.
10. **Is it a confidence interval?** No; it is a predictive conditional quantile estimate.

17. Practice Tasks

- Implement pinball loss for several tau values.
- Train 0.1, 0.5, 0.9 gradient-boosting quantile models.
- Measure interval coverage and average width.
- Find and visualize quantile crossing.
- Map an asymmetric business cost ratio to a candidate tau.

18. Project Ideas

Project	Stack/data	Resume value
Inventory safety-stock planner	sklearn; retail demand	Decision-aware uncertainty prediction.
Delivery SLA predictor	LightGBM/sklearn; delivery trips	Quantile-based risk communication.
Solar generation intervals	PyTorch; weather/solar data	Calibrated uncertainty evaluation.

19. Quick Revision

- **Idea:** predict percentile, not average; **formula:** asymmetric pinball loss.
- **Use:** asymmetric costs or intervals; **metric:** same pinball loss plus coverage.
- **Trap:** quantile is not exact-event probability; **one-liner:** "A 90th-quantile model deliberately overpredicts most observations."

20. Final Cheat Sheet

Definition	I/O and steps	Pros / cons	Best use
Asymmetric loss for conditional τ -quantile	vectors + $\tau \rightarrow$ residual, asymmetric linear penalty	decision-aware uncertainty / crossing and calibration challenges	intervals, SLAs, safety stock